

#### Μέθοδος Insert:

Η Insert δέχεται μια μεταβλητή τύπου LargeDepositor (item) για να το εισάγει στο δέντρο δυαδικής αναζήτησης. Για να γίνει η εισαγωγή καλεί τη insertAsRoot η οποία δέχεται ως ορίσματα την μεταβλητή που δόθηκε στην insert (item) και η μεταβλητή τύπου TreeNode (root) που έχει την ρίζα του δένδρου και βάση της τύχης επιλέγει αν το νέο στοιχείο θα γίνει ρίζα του δένδρου ή όχι . Στην περίπτωση που το δένδρο είναι κενό, η μεταβλητή item γίνεται η ρίζα αλλιώς, ανάλογα με την σύγκριση του AFM , επιλέγει την κατάλληλη κατεύθυνση για το item, ενώ ταυτόχρονα για την διατήρηση της ισορροπίας του δέντρου επιλέγει αν θα χρειαστεί να κάνει περαιτέρω περιστροφές στο δέντρο . Οι μέθοδοι περιστροφής του δέντρου, δηλαδή η rotateLeft() και η rotateRight() κάνουν αριστερή και δεξιά περιστροφή στους κόμβους του δέντρου και η rotateToRoot() μεταφέρει έναν κόμβο στη θέση της ρίζας κάνοντας τις αναγκαίες περιστροφές. Οι μέθοδοι Insert, insertAsRoot έχουν πολυπλοκότητα  $O(\log N)$

#### Μέθοδος load :

Η μέθοδος load διαβάζει δεδομένα από ένα αρχείο (στην συγκεκριμένη περίπτωση το filename.txt) διαχωρίζει τα δεδομένα με την μέθοδο split στον πίνακα data και από εκεί με τις κατάλληλες επεξεργασίες των τύπων των δεδομένων φτιάχνει αντικείμενο για να εισαχθεί το δυαδικό δέντρο. Έπειτα καλεί την insert και γίνεται ότι προαναφέρθηκε στην insert. Η πολυπλοκότητα της load είναι  $O(N * \log N)$  με N το πλήθος των Depositors στο αρχείο με την προϋπόθεση ότι το δέντρο είναι ισορροπημένο κατά την εισαγωγή.

#### Μέθοδος updateSavings :

Η μέθοδος updateSavings αναζητάει το ζητούμενο ΑΦΜ και ενημερώνει τα savings. Αρχικά καλείτε η updatesavingsLoop, η οποία πρώτα ελέγχει εάν η root είναι άδεια και εκτυπώνει ανάλογο μήνυμα. Έπειτα βλέπει εάν το ΑΦΜ που δώσαμε είναι ίσο με αυτού που ψάχνουμε, εάν είναι τότε ενημερώνει την τιμή του savings με την καινούργιά. Αν η ισότητα δεν ισχύει τότε ανάλογα με το ΑΦΜ να είναι μεγαλύτερο ή μικρότερο μπαίνει στο ανάλογο υποδέντρο και ξανακαλεί την updatesavingsLoop. Πρακτικά είναι σαν την διάδική αναζήτηση αρά θα έχει πολυπλοκότητα  $O(\log nN)$  όπου N ο αριθμός των κόμβων του δέντρου.

#### Μέθοδος searchByAFM:

Η μέθοδος searchByAFM επιστρέφει το αντικείμενο του ΑΦΜ που του δώσαμε. Παρομοίως με την updatesavings θα κάνουμε σχεδόν την ίδια μέθοδο, δηλαδή διαδική αναζήτηση, οπότε η searchByAFM θα καλεί την searchByAFMLoop. Κλασικά θα ελέγχουμε εάν η root είναι null για να δούμε εάν το δέντρο είναι άδειο. Έπειτα ελέγχει εάν το ΑΦΜ ισούται με το ΑΦΜ του αντικειμένου που ψάχνουμε, αν βρεθεί τότε τυπώνει τα στοιχεία του αντικειμένου και τελειώνει η μέθοδος. Αλλιώς μπαίνει στο ανάλογο υποδέντρο ανάλογα με το ΑΦΜ είναι μεγαλύτερο ή μικρότερο το

ΑΦΜ και ξανακαλείτε η `searchByAFMLoop`. Αυτή η μέθοδος είναι εξίσου  $O(\log N)$  με  $N$  τον αριθμό των κόμβων του δέντρου(διαδική αναζήτηση).

Μέθοδος `searchByLastName`:

Η Μέθοδος `searchByLastName` διασχίζει όλο το δέντρο για να αναζητήσει τα επώνυμα και να επιτρέψει τους καταθέτες. Η μέθοδος `searchByLastName` αρχικοποιεί την λίστα που θα έχει τα επώνυμα και ένα counter και της ελέγχει τη θα τυπώνει ενώ η υπολογισμούς και επαναλήψεις γίνονται στην `searchByLastNameLoop`. Η `searchByLastNameLoop` αρχικά αναζητά το αριστερό υποδέντρο για το ζητούμενο επώνυμο. Εάν βρεθεί επώνυμο προσθέτετε στην λίστα και ο μετρητής αυξάνετε κατά ένα και επαναλαμβάνετε και στο δεξί δέντρο έπειτα αν δεν έχουν βρεθεί ήδη 5 επώνυμα. Όταν βρεθούν 5 επώνυμα σταματάει η προσθήκη και εμφανίζει ανάλογο μήνυμα. Η πολυπλοκότητα της μεθόδου είναι  $O(N)$  όπου  $N$  είναι ο αριθμός των κόμβων του δέντρου καθώς μπορεί να περάσει από όλες της τιμές του δέντρου.

Μέθοδος `remove`:

Η μέθοδος `remove` αφαιρεί από το δέντρο τον Depositor με το αφμ που δίνουμε. Στην περίπτωση που το ΑΦΜ δεν βρεθεί, το πρόγραμμα σταματάει να εκτελείτε. Σε εναλλακτική περίπτωση, η μέθοδος καθορίζει τον parent κόμβο και στην συνέχεια καθορίζει με την `replace` τον κόμβο που θα αντικατασταθεί. Ωστόσο υπάρχουν 2 υποπερίπτώσεις σε αυτό. Η πρώτη περίπτωση είναι εάν κόμβος έχει κανένα ή μόνο ένα παιδί που σε αυτήν την περίπτωση, αν δεν έχει παιδί, παίρνει την τιμή null αλλιώς ως `replace` χρησιμοποιεί το παιδί τού. Στην δεύτερη περίπτωση, ο κόμβος από το δεξί υποδέντρο (δεξί παιδί) αντικαθιστάτε με τον κόμβο. Όποια και να ήταν η περίπτωση μας, στο τέλος αντικαθιστούμε τον κόμβο με τον κόμβο που έχουμε διαλέξει ως αντικαταστάτη. Η μέθοδος `remove` έχει περιπλοκότητα  $O(\log N)$

Μέθοδος `getMeanSavings`:

Η μέθοδος `getMeanSavings` επιστρέφει τον μέσο όρο όλων των καταθέσεων. Αρχικοποιώντας το `sumSavings` αι το  $N$  μ 0 έπειτα καλεί την βοηθητική μέθοδο `inOrderTraversal` η οποία δέχεται το `root`, το `sumSavings` και το  $N$  ως παραμέτρους.

Αν το `root` δεν είναι null τότε κάνει αναζήτηση σε όλη το δέντρο και αποθηκεύει τα `savings` όλων των καταθέσεων και προσθέτει +1 κάθε φορά που κάνει την πάνω πράξη και ανακαλεί τον εαυτό της. Στο τέλος, επιστρέφει στην `getMeanSavings` αν το  $N$  είναι θετικό επιστρέφει το μέσο όρο των καταθέσεων αλλιώς επιστρέφει 0 γιατί άρα το `root` είναι null και δεν υπάρχου καταθέτες και για να αποφευχθεί η διαίρεση και με το μηδέν. Η πολυπλοκότητα της `getMeanSavings` είναι  $O(N)$

#### Μέθοδος printTopLargeDepositos:

Η `printTopLargeDepositors` εκτυπώνει του top k καταθέτες που είναι πιο πιθανόν να φοροδιαφεύγουν όπου k δίνεται από τον χρήστη. Αρχικά ελέγχουμε αν το `root` είναι null και αν ισχύει εκτυπώνουμε κατάλληλο μήνυμα και βγαίνουμε από την μέθοδο. Σε εναλλακτική περίπτωση, δημιουργούμε τον πίνακα `topDepositors` με k θέσεις όπου θα τον χρησιμοποιήσουμε για να βρούμε τους top k που πιθανόν φοροδιαφεύγουν. Έπειτα καλούμε την `initTopDepositors` που παίρνει ως παράμετρος το `root`, τον πίνακα `topDepositors`, μια `int index` που είναι ίση με το 0 και το k. Έπειτα ψάχνουμε το δέντρο για να τους βρούμε που στην περίπτωση που Αν το `taxedIncome` είναι κάτω του 8000 τότε μπαίνει αμέσως στον πίνακα εναλλακτικά ελέγχουμε την των `Savings` πλην το `taxedIncome` και σε περίπτωση που δεν έχει γεμίσει ο πίνακας με την πάνω περίπτωση και είναι στις τις top k-index του δένδρου τότε μπαίνει στον πίνακα. Τέλος αν ο πίνακας είναι γεμάτος εκτυπώνουμε τον πίνακα `topDepositors` αλλιώς εκτυπώνουμε κατάλληλο μήνυμα ότι υπάρχουν λιγότερα από k στοιχεία στον πίνακα. Η πολυπλοκότητα του `printTopLargeDepositors` είναι  $O(N)$ .

#### Μέθοδος printByAFM:

Η `printByAFM` εκτυπώνει τα στοιχεία του δέντρου κατά αύξουσα σειρά βάση του ΑΦΜ, καλώντας την `inOrderTraversal`. Αφού η `inOrderTraversal` έχει πολυπλοκότητα  $O(N)$  τότε και η `printByAFM` έχει πολυπλοκότητα  $O(N)$